

艾米地产项目人工智能调度管理系统

程序鉴别材料（源代码）

版本：V1.0

本项目共包含 220 个核心源代码文件。

涵盖以下核心层：- 核心业务逻辑层 (emily-core/emily_core) - API 服务层 (emily-core/api) - 系统脚本层 (scripts)
- 插件适配层 (data/plugins/emily_agent)

文件目录索引

1. emily-core/emily_core/adapters/session/session_config.py
2. emily-core/emily_core/adapters/session/session_factory.py
3. emily-core/emily_core/adapters/session/session_pool.py
4. emily-core/emily_core/adapters/standard/command.py
5. emily-core/emily_core/adapters/standard/message.py
6. emily-core/emily_core/adapters/standard/reply.py
7. emily-core/emily_core/adapters/standard/result.py
8. emily-core/emily_core/adapters/standard/route_decision.py
9. emily-core/emily_core/agent/sop_parser.py
10. emily-core/emily_core/application/_user_utils.py
11. emily-core/emily_core/application/event_app.py
12. emily-core/emily_core/application/file_app.py
13. emily-core/emily_core/application/meeting_app.py
14. emily-core/emily_core/application/node_app.py
15. emily-core/emily_core/application/permission_app.py
16. emily-core/emily_core/application/query_app.py
17. emily-core/emily_core/application/task_app.py
18. emily-core/emily_core/bootstrap.py
19. emily-core/emily_core/infrastructure/database/models.py
20. emily-core/emily_core/infrastructure/database/scripts/fill_users_required_fields.py
21. emily-core/emily_core/infrastructure/database/session.py
22. emily-core/emily_core/infrastructure/llm/client.py
23. emily-core/emily_core/infrastructure/llm/prompt_loader.py
24. emily-core/emily_core/infrastructure/logging/business_event_logger.py
25. emily-core/emily_core/infrastructure/logging/feedback_detector.py
26. emily-core/emily_core/infrastructure/logging/legacy_log_bridge.py
27. emily-core/emily_core/infrastructure/logging/llm_logger.py
28. emily-core/emily_core/infrastructure/logging/log_writer.py
29. emily-core/emily_core/infrastructure/logging/pipeline_logger.py
30. emily-core/emily_core/infrastructure/logging/rag_logger.py

31. emily-core/emily_core/infrastructure/logging/scheduler_logger.py
32. emily-core/emily_core/infrastructure/logging/session_lifecycle_logger.py
33. emily-core/emily_core/node_event_bus.py
34. emily-core/emily_core/outbound_bus.py
35. emily-core/emily_core/permission/auth_engine.py
36. emily-core/emily_core/permission/cache.py
37. emily-core/emily_core/permission/code_compiler.py
38. emily-core/emily_core/permission/level.py
39. emily-core/emily_core/permission/row_security.py
40. emily-core/emily_core/providers/email/base.py
41. emily-core/emily_core/providers/email/imap_provider.py
42. emily-core/emily_core/providers/email/smtp_provider.py
43. emily-core/emily_core/providers/rag/base.py
44. emily-core/emily_core/providers/rag/local_fallback.py
45. emily-core/emily_core/providers/rag/maxkb_provider.py
46. emily-core/emily_core/repositories/agent_reasoning_repo.py
47. emily-core/emily_core/repositories/chat_archive_repo.py
48. emily-core/emily_core/repositories/event_repo.py
49. emily-core/emily_core/repositories/evolution_repo.py
50. emily-core/emily_core/repositories/file_repo.py
51. emily-core/emily_core/repositories/llm_interaction_repo.py
52. emily-core/emily_core/repositories/meeting_repo.py
53. emily-core/emily_core/repositories/message_repo.py
54. emily-core/emily_core/repositories/node_repo.py
55. emily-core/emily_core/repositories/permission_grant_repo.py
56. emily-core/emily_core/repositories/permission_repo.py
57. emily-core/emily_core/repositories/scheduler_repo.py
58. emily-core/emily_core/repositories/session_accessible_file_repo.py
59. emily-core/emily_core/repositories/session_archive_repo.py
60. emily-core/emily_core/repositories/system_description_repo.py
61. emily-core/emily_core/repositories/task_repo.py
62. emily-core/emily_core/repositories/tool_call_repo.py
63. emily-core/emily_core/repositories/tool_registry_repo.py
64. emily-core/emily_core/repositories/user_repo.py
65. emily-core/emily_core/repositories/world_book_repo.py
66. emily-core/emily_core/scheduler/application.py
67. emily-core/emily_core/scheduler/commands.py
68. emily-core/emily_core/scheduler/engine.py
69. emily-core/emily_core/scheduler/handler_registry.py
70. emily-core/emily_core/scheduler/hook_registry.py
71. emily-core/emily_core/scheduler/jobs/daily_insight.py
72. emily-core/emily_core/scheduler/jobs/data_sync.py
73. emily-core/emily_core/scheduler/jobs/health_check.py
74. emily-core/emily_core/scheduler/jobs/morning_report.py

- 75. emily-core/emily_core/scheduler/jobs/node_deadlines.py
- 76. emily-core/emily_core/scheduler/jobs/patch_validator.py
- 77. emily-core/emily_core/scheduler/jobs/periodic_node.py
- 78. emily-core/emily_core/scheduler/jobs/rule_induction.py
- 79. emily-core/emily_core/scheduler/jobs/session_cleanup.py
- 80. emily-core/emily_core/scheduler/jobs/system_description_update.py
- 81. emily-core/emily_core/scheduler/jobs/webhook.py
- 82. emily-core/emily_core/scheduler/jobs/world_book_update.py
- 83. emily-core/emily_core/scheduler/service.py
- 84. emily-core/emily_core/services/agent_trace_service.py
- 85. emily-core/emily_core/services/chat_archive_service.py
- 86. emily-core/emily_core/services/cognition_drift_detector.py
- 87. emily-core/emily_core/services/domain_takeover_service.py
- 88. emily-core/emily_core/services/email_service.py
- 89. emily-core/emily_core/services/event_journal.py
- 90. emily-core/emily_core/services/event_service.py
- 91. emily-core/emily_core/services/evolution/insight_generator.py
- 92. emily-core/emily_core/services/evolution/morning_report_builder.py
- 93. emily-core/emily_core/services/evolution/patch_applier.py
- 94. emily-core/emily_core/services/evolution/patch_generator.py
- 95. emily-core/emily_core/services/evolution/patch_validator.py
- 96. emily-core/emily_core/services/evolution/rule_inductor.py
- 97. emily-core/emily_core/services/file_service.py
- 98. emily-core/emily_core/services/file_storage_service.py
- 99. emily-core/emily_core/services/initialization_checker.py
- 100. emily-core/emily_core/services/meeting_service.py
- 101. emily-core/emily_core/services/message_service.py
- 102. emily-core/emily_core/services/node_batch.py
- 103. emily-core/emily_core/services/node_batch_update.py
- 104. emily-core/emily_core/services/node_commands.py
- 105. emily-core/emily_core/services/node_service.py
- 106. emily-core/emily_core/services/node_state_machine.py
- 107. emily-core/emily_core/services/pending_issues.py
- 108. emily-core/emily_core/services/permission_service.py
- 109. emily-core/emily_core/services/query_service.py
- 110. emily-core/emily_core/services/rule_book_loader.py
- 111. emily-core/emily_core/services/schema_drift_detector.py
- 112. emily-core/emily_core/services/system_description_builder.py
- 113. emily-core/emily_core/services/system_description_service.py
- 114. emily-core/emily_core/services/task_service.py
- 115. emily-core/emily_core/services/user_binding_service.py
- 116. emily-core/emily_core/services/user_memory_service.py
- 117. emily-core/emily_core/services/world_book_builder.py
- 118. emily-core/emily_core/services/world_book_service.py

- 119. emily-core/emily_core/session/confirm_queue.py
- 120. emily-core/emily_core/session/fetchers/fetch_available_tools.py
- 121. emily-core/emily_core/session/fetchers/fetch_rag_info.py
- 122. emily-core/emily_core/session/fetchers/fetch_system_description.py
- 123. emily-core/emily_core/session/fetchers/fetch_visible_files.py
- 124. emily-core/emily_core/session/fetchers/fetch_visible_schema.py
- 125. emily-core/emily_core/session/focus_lock.py
- 126. emily-core/emily_core/session/session_agent.py
- 127. emily-core/emily_core/session/session_context.py
- 128. emily-core/emily_core/session/session_data_fetcher.py
- 129. emily-core/emily_core/session/session_state.py
- 130. emily-core/emily_core/skill/definition.py
- 131. emily-core/emily_core/skill/executor.py
- 132. emily-core/emily_core/skill/param_extractor.py
- 133. emily-core/emily_core/skill/parser.py
- 134. emily-core/emily_core/skill/registry.py
- 135. emily-core/emily_core/skill/validator.py
- 136. emily-core/emily_core/tools/business_flow_tools.py
- 137. emily-core/emily_core/tools/chat_archive_tool.py
- 138. emily-core/emily_core/tools/definitions.py
- 139. emily-core/emily_core/tools/email_tool.py
- 140. emily-core/emily_core/tools/event_tool.py
- 141. emily-core/emily_core/tools/file_tool.py
- 142. emily-core/emily_core/tools/knowledge_search_tool.py
- 143. emily-core/emily_core/tools/meeting_tool.py
- 144. emily-core/emily_core/tools/memory_tool.py
- 145. emily-core/emily_core/tools/node_task_tool.py
- 146. emily-core/emily_core/tools/node_tool.py
- 147. emily-core/emily_core/tools/node_voice_entry_tool.py
- 148. emily-core/emily_core/tools/pending_issue_tool.py
- 149. emily-core/emily_core/tools/query_tool.py
- 150. emily-core/emily_core/tools/registry.py
- 151. emily-core/emily_core/tools/scripts/search_files.py
- 152. emily-core/emily_core/tools/task_tool.py
- 153. emily-core/emily_core/workitem/injector.py
- 154. emily-core/emily_core/workitem/pipeline/bus.py
- 155. emily-core/emily_core/workitem/pipeline/context.py
- 156. emily-core/emily_core/workitem/pipeline/hook.py
- 157. emily-core/emily_core/workitem/pipeline/hook_registry.py
- 158. emily-core/emily_core/workitem/pipeline/interfaces/auth.py
- 159. emily-core/emily_core/workitem/pipeline/interfaces/execution.py
- 160. emily-core/emily_core/workitem/pipeline/interfaces/guardian.py
- 161. emily-core/emily_core/workitem/pipeline/interfaces/planning.py
- 162. emily-core/emily_core/workitem/pipeline/interfaces/risk.py

163. emily-core/emily_core/workitem/pipeline/interfaces/routing.py
164. emily-core/emily_core/workitem/pipeline/mocks/mock_execution.py
165. emily-core/emily_core/workitem/pipeline/mocks/mock_planning.py
166. emily-core/emily_core/workitem/pipeline/node.py
167. emily-core/emily_core/workitem/pipeline/real_guardian.py
168. emily-core/emily_core/workitem/scheduler.py
169. emily-core/emily_core/workitem/workitem.py
170. emily-core/emily_core/workitem/workitem_agent.py
171. emily-core/emily_core/workitem/workitem_state.py
172. emily-core/api/middleware/auth.py
173. emily-core/api/routes/evolution.py
174. emily-core/api/routes/health.py
175. emily-core/api/routes/message.py
176. emily-core/api/routes/meta_cognition.py
177. emily-core/api/routes/node.py
178. emily-core/api/routes/node_schemas.py
179. emily-core/api/routes/permission.py
180. emily-core/api/routes/session.py
181. emily-core/api/routes/skills.py
182. emily-core/api/server.py
183. emily-core/api/sse/node_events.py
184. emily-core/api/sse/outbound.py
185. scripts/build_system_description.py
186. scripts/build_world_book.py
187. scripts/check_initialization.py
188. scripts/cognition_cycle.py
189. scripts/cold_start.py
190. scripts/collect_session_data.py
191. scripts/detect_cognition_drift.py
192. scripts/evolution.py
193. scripts/evolution_anomaly.py
194. scripts/evolution_apply.py
195. scripts/evolution_insight.py
196. scripts/evolution_metrics.py
197. scripts/evolution_morning.py
198. scripts/evolution_node_close.py
199. scripts/evolution_patch.py
200. scripts/evolution_rules.py
201. scripts/evolution_validate.py
202. scripts/generate_session_prompt.py
203. scripts/manage_nodes.py
204. scripts/register_api.py
205. scripts/rescan_files.py
206. scripts/self_check.py

207. scripts/sop_to_skill.py
208. scripts/update_world_book.py
209. data/plugins/emily_agent/_conf_schema.json
210. data/plugins/emily_agent/adapters/api_client.py
211. data/plugins/emily_agent/adapters/astrbot/inbound_adapter.py
212. data/plugins/emily_agent/adapters/astrbot/outbound_sender.py
213. data/plugins/emily_agent/adapters/sse_listener.py
214. data/plugins/emily_agent/adapters/standard/command.py
215. data/plugins/emily_agent/adapters/standard/message.py
216. data/plugins/emily_agent/adapters/standard/reply.py
217. data/plugins/emily_agent/adapters/standard/result.py
218. data/plugins/emily_agent/adapters/standard/route_decision.py
219. data/plugins/emily_agent/main.py
220. data/plugins/emily_agent/metadata.yaml

文件编号：1 文件路径：emily-core/emily_core/adapters/session/session_config.py

```
from __future__ import annotations
from dataclasses import dataclass
@dataclass
class SessionConfig:
    ttl_seconds: int = 600
    max_concurrent: int = 100
    max_workitems_per_session: int = 5
    sweep_interval_seconds: int = 60
    @classmethod
    def from_config(cls, config) -> "SessionConfig":
        return cls(
            ttl_seconds=getattr(config, "session_ttl_seconds", 600),
            max_concurrent=getattr(config, "session_max_concurrent", 100),
            max_workitems_per_session=getattr(config, "workitem_max_per_session", 5),
        )
```

文件编号：2 文件路径：emily-core/emily_core/adapters/session/session_factory.py

```
from __future__ import annotations
import logging
from typing import TYPE_CHECKING
from ...session.session_agent import SessionAgent
from ...session.session_context import SessionContext
if TYPE_CHECKING:
    from ...adapters.standard.message import StandardMessage
    from ...workitem.pipeline.bus import PipelineBUS
logger = logging.getLogger("emily.session_factory")
class SessionFactory:
    def __init__(self, bus: "PipelineBUS", core=None):
```

```
self._bus = bus
self._core = core
def create(self, message: "StandardMessage", user_id: str = "") -> SessionAgent:
    conv_id = message.conversation_id
    context = self._build_context(message, user_id)
    llm = None
    skill_registry = None
    if self._core is not None:
        llm = getattr(self._core, "_llm_client", None)
        skill_registry = getattr(self._core, "_skill_registry", None)
    agent = SessionAgent(
        conversation_id=conv_id,
        context=context,
        bus=self._bus,
        llm_client=llm,
        skill_registry=skill_registry,
    )
    logger.info(
        "SessionFactory created session: conv=%s user=%s llm=%s skill_registry=%s",
        conv_id, user_id or "?",
        "yes" if llm else "no",
        "yes" if skill_registry else "no",
    )
    return agent
def _build_context(self, message: "StandardMessage", user_id: str) -> SessionContext:
    return SessionContext.create(
        user_id=user_id,
        conversation_id=message.conversation_id,
        sender_name=message.sender_name or "",
        core=self._core,
    )
```

文件编号：3 文件路径：emily-core/emily_core/adapters/session/session_pool.py

```
from __future__ import annotations
import asyncio
import logging
import time
from typing import TYPE_CHECKING
from .session_config import SessionConfig
from .session_factory import SessionFactory
if TYPE_CHECKING:
    from ...adapters.standard.message import StandardMessage
    from ...adapters.standard.reply import ReplyMessage
    from ...session.session_agent import SessionAgent
```

```
from ...workitem.pipeline.bus import PipelineBUS
logger = logging.getLogger("emily.session_pool")
class _Entry:
    __slots__ = ("agent", "last_active", "lock")
    def __init__(self, agent):
        self.agent = agent
        self.last_active = time.time()
        self.lock = asyncio.Lock()
class SessionPoolManager:
    def __init__(
        self,
        bus: "PipelineBUS",
        config: SessionConfig | None = None,
        factory: SessionFactory | None = None,
        core=None,
    ):
        self._config = config or SessionConfig()
        self._factory = factory or SessionFactory(bus, core=core)
        self._sessions: dict[str, _Entry] = {}
        self._start_time = time.time()
        self._sweeper_task: asyncio.Task | None = None
    async def _start_sweeper(self) -> None:
        if self._sweeper_task is not None:
            return
        interval = getattr(self._config, "sweep_interval_seconds", 300) or 300
        self._sweeper_task = asyncio.create_task(self._sweep_loop(interval))
        logger.info("SessionPool sweeper started (interval=%ds)", interval)
    async def _sweep_loop(self, interval: int) -> None:
        while True:
            await asyncio.sleep(interval)
            try:
                removed = self.sweep_expired()
                if removed:
                    logger.debug("SessionPool sweeper removed %d expired sessions", removed)
            except Exception as e:
                logger.warning("SessionPool sweeper error: %s", e)
    async def _ensure_sweeper(self) -> None:
        if self._sweeper_task is None:
            await self._start_sweeper()
    async def route(self, message: "StandardMessage", user_id: str = "") -> "ReplyMessage | None":
        conv_id = message.conversation_id
        entry = self._sessions.get(conv_id)
        await self._ensure_sweeper()
        if entry is None:
```



```
    if len(self._sessions) >= self._config.max_concurrent:
        self.sweep_expired()
        agent = self._factory.create(message, user_id=user_id)
        entry = _Entry(agent)
        self._sessions[conv_id] = entry
        logger.info("SessionPool created: conv=%s (pool size=%d)",
                    conv_id, len(self._sessions))
    else:
        logger.debug("SessionPool hit: conv=%s", conv_id)
    async with entry.lock:
        entry.last_active = time.time()
        return await entry.agent.handle(message)
def lookup(self, conversation_id: str) -> "SessionAgent | None":
    entry = self._sessions.get(conversation_id)
    return entry.agent if entry else None
async def terminate(self, conversation_id: str) -> bool:
    entry = self._sessions.get(conversation_id)
    if entry is None:
        return False
    try:
        await entry.agent.archive()
    except Exception as e:
        logger.warning("SessionPool terminate archive failed: %s", e)
    self._sessions.pop(conversation_id, None)
    logger.info("SessionPool terminated: conv=%s", conversation_id)
    return True
def sweep_expired(self) -> int:
    now = time.time()
    ttl = self._config.ttl_seconds
    expired = [
        cid for cid, e in self._sessions.items()
        if now - e.last_active > ttl
    ]
    for cid in expired:
        entry = self._sessions.pop(cid, None)
        if entry:
            try:
                asyncio.ensure_future(entry.agent.archive())
            except Exception as e:
                logger.warning("SessionPool sweep archive failed for %s: %s", cid, e)
    if expired:
        logger.info("SessionPool swept %d expired session(s)", len(expired))
    return len(expired)
@property
```

```
def size(self) -> int:
    return len(self._sessions)
@property
def uptime_seconds(self) -> int:
    return int(time.time() - self._start_time)
```

文件编号：4 文件路径：emily-core/emily_core/adapters/standard/command.py

```
from dataclasses import dataclass
@dataclass
class EventCommand:
    project_id: str | None = None
    project_name: str | None = None
    title: str = ""
    event_type: str = "general"
    category: str = "待分类"
    description: str = ""
    event_date: str | None = None
    creator_id: str = ""
    source_message_id: str = ""
    related_event_ids: list[str] | None = None
    conversation_id: str = ""
@dataclass
class TaskCommand:
    project_id: str | None = None
    project_name: str | None = None
    title: str = ""
    description: str = ""
    assignee_text: str = ""
    due_date: str | None = None
    due_text: str = ""
    creator_id: str = ""
    source_message_id: str = ""
@dataclass
class MeetingCommand:
    project_id: str | None = None
    project_name: str | None = None
    title: str = ""
    summary: str = ""
    attendees: list[str] | None = None
    creator_id: str = ""
    source_message_id: str = ""
@dataclass
class FileCommand:
    project_id: str | None = None
```

```
project_name: str | None = None
filename: str = ""
file_type: str = ""
file_size: int = 0
storage_path: str = ""
file_category: str = "OTHER"
uploaded_by: str = ""
source_message_id: str = ""
```

@dataclass

```
class QueryCommand:
    query_type: str = "event"
    project_id: str | None = None
    project_ids: list[str] | None = None
    project_name: str | None = None
    time_range: str = "all"
    status_filter: str | None = None
    assignee: str | None = None
    sender_name: str | None = None
    keyword: str | None = None
    intent: str | None = None
    file_type: str | None = None
    conversation_id: str | None = None
    limit: int = 50
```

文件编号: 5 文件路径: emily-core/emily_core/adapters/standard/message.py

```
from dataclasses import dataclass, field
```

```
from datetime import datetime
```

@dataclass

```
class StandardMessage:
    message_id: str = ""
    platform: str = ""
    conversation_type: str = ""
    conversation_id: str = ""
    sender_id: str = ""
    sender_name: str = ""
    group_id: str | None = None
    group_name: str | None = None
    content: str = ""
    is_at_bot: bool = False
    mentioned_user_ids: list[str] = field(default_factory=list)
    reply_to_message_id: str | None = None
    timestamp: datetime | None = None
    raw_event: dict | None = None
    msg_type: int = 1
```

```
attachments: list[dict] = field(default_factory=list)
```

文件编号: 6 文件路径: emily-core/emily_core/adapters/standard/reply.py

```
from dataclasses import dataclass, field
@dataclass
class ReplyMessage:
    conversation_id: str
    content: str
    reply_to_message_id: str | None = None
    references: list[dict] = field(default_factory=list)
    metadata: dict = field(default_factory=dict)
    file_paths: list[dict] = field(default_factory=list)
```

文件编号: 7 文件路径: emily-core/emily_core/adapters/standard/result.py

```
from dataclasses import dataclass, field
@dataclass
class RouteResult:
    intent: str
    project_id: str | None = None
    project_name: str | None = None
    confidence: float = 0.0
    data: dict = field(default_factory=dict)
@dataclass
class HandlerResult:
    success: bool
    object_type: str | None = None
    object_id: str | None = None
    reply: str | None = None
    error_code: str | None = None
    pending_confirmation: bool = False
@dataclass
class AgentStep:
    step_index: int
    type: str
    detail: str
    timestamp: str = ""
@dataclass
class AgentResult:
    success: bool
    reply: str
    steps: list[AgentStep] = field(default_factory=list)
    error_code: str | None = None
```

文件编号: 8 文件路径: emily-core/emily_core/adapters/standard/route_decision.py

```
from dataclasses import dataclass
@dataclass
class RouteDecision:
    takeover: bool
    mode: str = "collaborate"
    intent: str | None = None
    confidence: float = 0.0
    handler: str | None = None
    should_reply: bool = True
    reason: str = ""
```

文件编号: 9 文件路径: emily-core/emily_core/agent/sop_parser.py

```
import re
from typing import Any
_md: Any = None
def _get_md() -> Any:
    global _md
    if _md is None:
        import mistune # type: ignore[import-untyped]
        _md = mistune.create_markdown(renderer="ast", plugins=["table"])
    return _md
def _parse_to_tokens(content: str) -> List[dict[str, Any]]:
    return _get_md()(content) # type: ignore[no-any-return]
def _extract_text(children: List[dict[str, Any]]) -> str:
    parts: List[str] = []
    for child in children:
        ttype = child.get("type", "")
        if ttype == "text":
            parts.append(child.get("raw", ""))
        elif ttype == "codespan":
            attrs = child.get("attrs", {})
            parts.append(attrs.get("text", child.get("raw", "")))
        elif ttype in (
            "strong", "emphasis", "link", "image",
            "block_text", "block_code", "paragraph",
        ):
            parts.append(_extract_text(child.get("children", [])))
        elif ttype in ("softbreak", "linebreak"):
            parts.append(" ")
        elif child.get("children"):
            parts.append(_extract_text(child["children"]))
    return "".join(parts)
def _extract_codespan_values(children: List[dict[str, Any]]) -> List[str]:
    codes: List[str] = []
```

```
for child in children:
    ttype = child.get("type", "")
    if ttype == "codespan":
        attrs = child.get("attrs", {})
        codes.append(attrs.get("text", child.get("raw", "")))
    elif child.get("children"):
        codes.extend(_extract_codespan_values(child["children"]))
return codes

def _has_strong_prefix(children: list[dict[str, Any]], text_prefix: str) -> bool:
    for child in children:
        if child.get("type") == "strong":
            txt = _extract_text(child.get("children", []))
            if txt.startswith(text_prefix):
                return True
    return False

def _collect_paragraph_texts(children: list[dict[str, Any]]) -> list[str]:
    lines: list[str] = []
    for child in children:
        if child.get("type") == "paragraph":
            lines.append(_extract_text(child.get("children", [])))
        elif child.get("children"):
            lines.extend(_collect_paragraph_texts(child["children"]))
    return lines

class _SOPBlockWalker:
    def __init__(self) -> None:
        self.h2_num: int | None = None
        self.h3_id: str | None = None
        self.in_must: bool = False
        self.in_deny: bool = False
        self.in_pos_example: bool = False
        self.in_neg_example: bool = False
        self.display_name: str = ""
        self.sop_id: str = ""
        self.sop_type: str = ""
        self.version: str = ""
        self.allow_roles: list[str] = []
        self.trigger_keywords: list[str] = []
        self.deny_conditions: list[str] = []
        self.positive_examples: list[str] = []
        self.negative_examples: list[str] = []
        self.allowed_tools: list[str] = []
    def walk(self, tokens: list[dict[str, Any]]) -> dict[str, Any]:
        for token in tokens:
            ttype: str = token.get("type", "")
```

```
    if ttype == "heading":
        self._on_heading(token)
    elif ttype == "table":
        self._on_table(token)
    elif ttype == "list":
        self._on_list(token)
    elif ttype == "block_quote":
        self._on_block_quote(token)
    elif ttype == "paragraph":
        self._on_paragraph(token)
return {
    "display_name": self.display_name,
    "sop_id": self.sop_id,
    "sop_type": self.sop_type,
    "version": self.version,
    "allow_roles": tuple(self.allow_roles) if self.allow_roles else (),
    "trigger_keywords": tuple(self.trigger_keywords),
    "deny_conditions": tuple(self.deny_conditions),
    "positive_examples": tuple(self.positive_examples),
    "negative_examples": tuple(self.negative_examples),
    "allowed_tools": list(self.allowed_tools),
}
def _on_heading(self, token: dict[str, Any]) -> None:
    children: list[dict[str, Any]] = token.get("children", [])
    text = _extract_text(children).strip()
    level: int = token.get("attrs", {}).get("level", 0)
    if level == 1:
        name = re.sub(r"/s*—/s* 业务流服务手册/s*$", "", text)
        self.display_name = name.strip()
    elif level == 2:
        m = re.match(r"/(d+)/.", text)
        self._h2_num = int(m.group(1)) if m else None
        self._h3_id = None
        self._reset_section_state()
    elif level == 3:
        m = re.match(r"/(d+/.+/d+)", text)
        self._h3_id = m.group(1) if m else None
        self._reset_section_state()
def _on_table(self, token: dict[str, Any]) -> None:
    if self._h2_num == 1:
        self._extract_chapter1_table(token)
    elif self._h2_num == 3 and self._h3_id == "3.2":
        self._extract_section_32_table(token)
def _on_list(self, token: dict[str, Any]) -> None:
```

```

    if self._h2_num != 2:
        return
    if self._h3_id not in ("2.1", None):
        return
    for item_token in token.get("children", []):
        if item_token.get("type") != "list_item":
            continue
        item_text = _extract_text(item_token.get("children", [])).strip()
        if not item_text:
            continue
        if self._in_must:
            sub_items = re.split(r"[,; ;]", item_text)
            for sub in sub_items:
                sub = sub.strip()
                if sub and len(sub) > 2:
                    self.trigger_keywords.append(sub)
        elif self._in_deny:
            self.deny_conditions.append(item_text)
def _on_block_quote(self, token: dict[str, Any]) -> None:
    if self._h2_num != 2 or self._h3_id != "2.2":
        return
    lines = _collect_paragraph_texts(token.get("children", []))
    if not lines:
        return
    for line_text in lines:
        line_text = line_text.strip()
        if not line_text:
            continue
        if self._in_pos_example:
            quotes = re.findall(r"「([^\"]+)」", line_text)
            for q in quotes:
                q = q.strip()
                if q:
                    self.positive_examples.append(q)
        elif self._in_neg_example:
            quotes = re.findall(r"「([^\"]+)」", line_text)
            for q in quotes:
                q = q.strip()
                if q:
                    self.negative_examples.append(q)
        arrows = re.findall(r"「/s*→/s*(.+?)」(?=「|$)」", line_text)
        for a in arrows:
            a = a.strip()
            if a and a not in self.negative_examples:

```



```
        self.negative_examples.append(a)
def _on_paragraph(self, token: dict[str, Any]) -> None:
    children: list[dict[str, Any]] = token.get("children", [])
    if _has_strong_prefix(children, " 必须条件"):
        self._in_must = True
        self._in_deny = False
    elif _has_strong_prefix(children, " 否定条件"):
        self._in_must = False
        self._in_deny = True
    elif _has_strong_prefix(children, " 应触发此业务流"):
        self._in_pos_example = True
        self._in_neg_example = False
    elif _has_strong_prefix(children, " 不应触发此业务流"):
        self._in_pos_example = False
        self._in_neg_example = True
def _extract_chapter1_table(self, token: dict[str, Any]) -> None:
    for section in token.get("children", []):
        for row_token in section.get("children", []):
            if row_token.get("type") != "table_row":
                continue
            cells: list[dict[str, Any]] = row_token.get("children", [])
            if len(cells) < 2:
                continue
            key = _extract_text(cells[0].get("children", [])).strip()
            val_children = cells[1].get("children", [])
            if key == " 业务流程编号" and not self.sop_id:
                self.sop_id = _extract_text(val_children).strip()
            elif key == " 版本" and not self.version:
                self.version = _extract_text(val_children).strip()
            elif key == " 权限控制":
                roles = _extract_codespan_values(val_children)
                seen: set[str] = set()
                unique: list[str] = []
                for r in roles:
                    if r not in seen:
                        seen.add(r)
                        unique.append(r)
                if unique:
                    self.allow_roles = unique
            elif key == " 业务类型" and not self.sop_type:
                self.sop_type = _extract_text(val_children).strip()
def _extract_section32_table(self, token: dict[str, Any]) -> None:
    for section in token.get("children", []):
        for row_token in section.get("children", []):
```

```

        if row_token.get("type") != "table_row":
            continue
        cells: List[dict[str, Any]] = row_token.get("children", [])
        if not cells:
            continue
        tool_names = _extract_codespan_values(cells[0].get("children", []))
        for name in tool_names:
            if (
                name.startswith(("record_", "submit_", "review_", "query_", "invoke_", "manage_", "write_"))
                and name not in self.allowed_tools:
                    self.allowed_tools.append(name)

    def _reset_section_state(self) -> None:
        self._in_must = False
        self._in_deny = False
        self._in_pos_example = False
        self._in_neg_example = False

    def parse_sop_markdown(content: str, file_path: str, file_name: str) -> dict[str, Any]:
        tokens = _parse_to_tokens(content)
        walker = _SOPBlockWalker()
        return walker.walk(tokens)

    def extract_allowed_tools_from_sop(sop_text: str) -> list[str]:
        tokens = _parse_to_tokens(sop_text)
        walker = _SOPBlockWalker()
        result = walker.walk(tokens)
        return result.get("allowed_tools", [])

```

文件编号：10 文件路径：emily-core/emily_core/application/_user_utils.py

```

import logging
logger = logging.getLogger("emily.app.user_utils")

def resolve_user_name(user_id: str) -> str:
    if not user_id:
        return ""
    try:
        from ..repositories.user_repo import UserRepository
        u = UserRepository.get(user_id)
        if u:
            return u.username or ""
    except Exception as e:
        logger.debug("resolve_user_name failed for %s: %s", user_id, e)
    return ""

```

文件编号：11 文件路径：emily-core/emily_core/application/event_app.py

```

import asyncio

```

```
import json
import logging
from typing import Optional
from ..adapters.standard.result import RouteResult, HandlerResult
from ..adapters.standard.command import EventCommand
from ..services.event_service import EventService
from ..infrastructure.database.models import Event
logger = logging.getLogger("emily.app.event")
def _log_business_event(**kwargs) -> None:
    try:
        from ..infrastructure.logging.business_event_logger import BusinessEventLogger
        ctx = BusinessEventLogger._current_context
        kwargs.setdefault("pipeline_run_id", ctx.get("pipeline_run_id", ""))
        kwargs.setdefault("conversation_id", ctx.get("conversation_id", ""))
        asyncio.ensure_future(BusinessEventLogger.log(**kwargs))
    except Exception:
        pass
class EventApplication:
    def __init__(self, event_service: EventService):
        self.event_service = event_service
        self._journal = None
    def set_journal(self, journal) -> None:
        self._journal = journal
    async def handle_event(
        self,
        route_result: RouteResult,
        user_id: str,
        message_id: str,
    ) -> HandlerResult:
        try:
            cmd = self._build_command(route_result, user_id, message_id)
            event = self.event_service.create_pending_event(cmd)
            project_name = route_result.project_name
            reply = EventService.format_confirmation_reply(event, project_name)
            _log_business_event(
                event_category="event",
                event_action="created",
                target_type="event",
                target_id=event.id,
                target_no=getattr(event, "event_no", "") or "",
                summary=f"创建事件: {event.title[:100]}",
                user_id=user_id,
                project_id=route_result.project_id or "",
            )
```

```
        return HandlerResult(
            success=True,
            object_type="event",
            object_id=event.id,
            reply=reply,
            pending_confirmation=True,
        )
    except Exception as e:
        logger.error("Event handling failed: %s", e, exc_info=True)
        return HandlerResult(
            success=False,
            error_code="event_create_failed",
            reply=f"事件录入失败: {e}",
        )
    def handle_confirmation(
        self,
        event_id: str,
        action: str,
    ) -> HandlerResult:
        if action == "confirm":
            event = self.event_service.confirm_event(event_id)
            if event:
                if self._journal is not None:
                    from ._user_utils import resolve_user_name
                    user_name = resolve_user_name(event.user_id) if event.user_id else ""
                    self._journal.append(
                        name=user_name or "用户",
                        summary=f"确认录入事件: {event.title} ({event.event_no})",
                    )
                _log_business_event(
                    event_category="event",
                    event_action="confirmed",
                    target_type="event",
                    target_id=event.id,
                    target_no=getattr(event, "event_no", "") or "",
                    summary=f"确认事件: {event.title[:100]}",
                    user_id=event.user_id or "",
                    project_id=getattr(event, "project_id", "") or "",
                )
            return HandlerResult(
                success=True,
                object_type="event",
                object_id=event.id,
                reply=f"☑ 已记录该事件 ({event.event_no})",
```

```
        )
    else:
        return HandlerResult(
            success=False,
            error_code="confirm_failed",
            reply=" 确认失败，事件不存在或已处理",
        )
    elif action == "cancel":
        event = self.event_service.cancel_event(event_id)
        if event:
            return HandlerResult(
                success=True,
                object_type="event",
                object_id=event.id,
                reply="☑ 已取消录入",
            )
        else:
            return HandlerResult(
                success=False,
                error_code="cancel_failed",
                reply=" 取消失败，事件不存在或已处理",
            )
    else:
        return HandlerResult(
            success=False,
            error_code="unknown_action",
            reply=f" 未知操作： {action}",
        )
)

@staticmethod
def _build_command(
    route_result: RouteResult,
    user_id: str,
    message_id: str,
) -> EventCommand:
    data = route_result.data or {}
    related = data.get("related_event_ids")
    if isinstance(related, str):
        import json as _json
        try:
            related = _json.loads(related)
        except (_json.JSONDecodeError, TypeError):
            related = [related] if related else []
    return EventCommand(
        project_id=route_result.project_id,
```

```
        project_name=route_result.project_name,
        title=data.get("title", " 未命名事件"),
        event_type=data.get("event_type", "general"),
        category=" 待分类",
        description=data.get("description", ""),
        event_date=data.get("event_date"),
        creator_id=user_id,
        source_message_id=message_id,
        related_event_ids=related if isinstance(related, list) else None,
        conversation_id=data.get("_conversation_id", ""),
    )
```

文件编号：12 文件路径：emily-core/emily_core/application/file_app.py

```
import logging
from ..adapters.standard.result import RouteResult, HandlerResult
from ..adapters.standard.command import FileCommand
from ..services.file_service import FileService
logger = logging.getLogger("emily.app.file")
class FileApplication:
    def __init__(self, file_service: FileService, storage_service=None):
        self.file_service = file_service
        self.storage_service = storage_service
        self._journal = None
    def set_journal(self, journal) -> None:
        self._journal = journal
    def set_storage_service(self, storage_service) -> None:
        self.storage_service = storage_service
    async def handle_file(
        self, route_result: RouteResult, user_id: str, message_id: str,
        attachment_url: str = "", attachment_type: int = 0,
        source_filename: str = "",
    ) -> HandlerResult:
        try:
            data = route_result.data or {}
            filename = data.get("filename", " 未命名文件")
            cmd = FileCommand(
                project_id=route_result.project_id,
                project_name=route_result.project_name,
                filename=filename,
                file_type=data.get("file_type", ""),
                uploaded_by=user_id,
                source_message_id=message_id,
                file_category=data.get("file_category", "OTHER"),
            )
```

```
f = self.file_service.create_file_record(cmd)
local_path = ""
if attachment_url and self.storage_service:
    try:
        store_result = self.storage_service.store_attachment(
            message_id=message_id,
            attachment_url=attachment_url,
            attachment_type=attachment_type or 3,
            source_filename=source_filename or filename,
        )
        if store_result:
            local_path = store_result.get("local_path", "")
            logger.info("File physically stored: %s", local_path)
    except Exception as e:
        logger.warning("Physical file storage failed (non-blocking): %s", e)
if self._journal is not None:
    from ._user_utils import resolve_user_name
    user_name = resolve_user_name(cmd.uploaded_by) or " 用户"
    summary = f" 归档文件: {f.filename} ({f.file_no}) "
    if local_path:
        summary += f" → {local_path}"
    self._journal.append(name=user_name, summary=summary)
reply = FileService.format_reply(f)
if local_path:
    reply += f"/n 文件已保存到本地存储。"
return HandlerResult(
    success=True, object_type="file", object_id=f.id, reply=reply,
)
except Exception as e:
    logger.error("File record creation failed: %s", e, exc_info=True)
return HandlerResult(
    success=False, error_code="file_create_failed", reply=f" 文件归档失败: {e}",
)
async def handle_list_by_category(
    self,
    file_category: str | None = None,
    project_id: str | None = None,
    project_ids: list[str] | None = None,
    keyword: str = "",
    limit: int = 10,
) -> HandlerResult:
    try:
        from ..infrastructure.database.models import FileCategory
        files = self.file_service.list_by_category(
```

```

        project_id=project_id,
        project_ids=project_ids,
        file_category=file_category,
        limit=limit,
    )
    if not files:
        cat_name = FileCategory.display(file_category) if file_category else " 文件"
        return HandlerResult(
            success=True,
            reply=f"☒ {cat_name} 类暂无文件记录。",
        )
    cat_name = FileCategory.display(file_category) if file_category else " 全部"
    lines = [f"☒ {cat_name} 类文件 (共 {len(files)} 份)", "_____"]
    for i, f in enumerate(files[:10], 1):
        cat_display = FileCategory.display(getattr(f, 'file_category', 'OTHER'))
        lines.append(f"{i}. {f.filename} ({f.file_no}) [{cat_display}]")
    if len(files) > 10:
        lines.append(f"... 还有 {len(files) - 10} 份")
    return HandlerResult(
        success=True,
        reply="/n".join(lines),
        data={
            "total": len(files),
            "category": file_category,
            "files": [
                {
                    "file_no": f.file_no,
                    "filename": f.filename,
                    "file_category": getattr(f, 'file_category', 'OTHER'),
                }
                for f in files[:20]
            ],
        },
    )
except Exception as e:
    logger.error("List files by category failed: %s", e, exc_info=True)
    return HandlerResult(
        success=False,
        error_code="file_query_failed",
        reply=f" 文件查询失败: {e}",
    )
async def handle_update_category(
    self,
    file_no: str,

```



```
        file_category: str,
        user_id: str = "",
    ) -> HandlerResult:
        try:
            from ..infrastructure.database.models import FileCategory
            f = self.file_service.repo.get_by_file_no(file_no)
            if f is None:
                return HandlerResult(
                    success=False,
                    reply=f"找不到文件编号 {file_no}",
                )
            old_category = getattr(f, 'file_category', 'OTHER')
            old_display = FileCategory.display(old_category)
            new_display = FileCategory.display(file_category)
            result = self.file_service.update_file_category(
                file_id=f.id,
                file_category=file_category,
                operator_id=user_id,
            )
            if result is None:
                return HandlerResult(
                    success=False,
                    reply=f"文件分类更新失败",
                )
            return HandlerResult(
                success=True,
                reply=f"☑ 文件「{f.filename}」已从「{old_display}」改到「{new_display}」",
                data={
                    "file_no": file_no,
                    "old_category": old_category,
                    "new_category": file_category,
                },
            )
        except Exception as e:
            logger.error("Update file category failed: %s", e, exc_info=True)
            return HandlerResult(
                success=False,
                error_code="file_category_update_failed",
                reply=f"分类更新失败: {e}",
            )
```

文件编号: 13 文件路径: emily-core/emily_core/application/meeting_app.py

```
import asyncio
import logging
```

```
from ..adapters.standard.result import RouteResult, HandlerResult
from ..adapters.standard.command import MeetingCommand
from ..services.meeting_service import MeetingService
logger = logging.getLogger("emily.app.meeting")
def _log_business_event(**kwargs) -> None:
    try:
        from ..infrastructure.logging.business_event_logger import BusinessEventLogger
        ctx = BusinessEventLogger._current_context
        kwargs.setdefault("pipeline_run_id", ctx.get("pipeline_run_id", ""))
        kwargs.setdefault("conversation_id", ctx.get("conversation_id", ""))
        asyncio.ensure_future(BusinessEventLogger.log(**kwargs))
    except Exception:
        pass
class MeetingApplication:
    def __init__(self, meeting_service: MeetingService):
        self.meeting_service = meeting_service
        self._journal = None
    def set_journal(self, journal) -> None:
        self._journal = journal
    async def handle_meeting(
        self, route_result: RouteResult, user_id: str, message_id: str
    ) -> HandlerResult:
        try:
            data = route_result.data or {}
            cmd = MeetingCommand(
                project_id=route_result.project_id,
                project_name=route_result.project_name,
                title=data.get("title", " 未命名会议"),
                summary=data.get("summary", ""),
                attendees=data.get("attendees") or [],
                creator_id=user_id,
                source_message_id=message_id,
            )
            meeting = self.meeting_service.create_meeting(cmd)
            if self._journal is not None:
                from ._user_utils import resolve_user_name
                user_name = resolve_user_name(cmd.creator_id) or " 用户"
                self._journal.append(
                    name=user_name,
                    summary=f" 录入会议纪要: {meeting.title} ({meeting.meeting_no}) ",
                )
            from ._user_utils import resolve_user_name
            _uname = resolve_user_name(cmd.creator_id) or ""
            _log_business_event(
```

```

        event_category="meeting",
        event_action="created",
        target_type="meeting",
        target_id=meeting.id,
        target_no=getattr(meeting, "meeting_no", "") or "",
        summary=f" 录入会议: {meeting.title[:100]}",
        user_id=user_id,
        user_name=_uname,
        project_id=route_result.project_id or "",
    )
    reply = MeetingService.format_reply(meeting)
    return HandlerResult(
        success=True, object_type="meeting", object_id=meeting.id, reply=reply,
    )
except Exception as e:
    logger.error("Meeting creation failed: %s", e, exc_info=True)
    return HandlerResult(
        success=False, error_code="meeting_create_failed", reply=f" 会议归档失败: {e}",
    )

```

文件编号: 14 文件路径: emily-core/emily_core/application/node_app.py

```

from __future__ import annotations
import logging
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from ..services.node_service import NodeService
    from ..services.node_commands import (
        CreateNodeCommand,
        UpdateNodeCommand,
        DiscardNodeCommand,
        ActivateNodeCommand,
        CreateDeliverableCommand,
        UpdateDeliverableProgressCommand,
        AddDependencyCommand,
        RemoveDependencyCommand,
        MountChildCommand,
        UnmountChildCommand,
    )
logger = logging.getLogger("emily.node_app")
class NodeApplication:
    def __init__(self, service: "NodeService", auth_engine=None):
        self._service = service
        self._auth_engine = auth_engine
    async def _check_node_permission(self, node_id: str, operator_id: str,

```

```

        operation: str = "write") -> bool:
    if not self._auth_engine or not operator_id:
        return True
    try:
        result = await self._auth_engine.check_access(
            perms=None,
            resource_type="NODE",
            resource_id=node_id,
            operation=operation,
        )
        return result.allowed
    except Exception:
        logger.warning("Permission check failed for node=%s user=%s", node_id, operator_id)
        return True
def _require_permission(self, allowed: bool, node_id: str, operation: str) -> None:
    if not allowed:
        raise PermissionError(f" 无权限对节点 {node_id} 执行 {operation} 操作")
async def create_node(self, cmd: "CreateNodeCommand") -> dict:
    result = await self._service.create_node(cmd)
    return {
        "success": result.success,
        "node_id": result.node_id,
        "status": result.status,
        "progress": result.progress,
        "reply": result.message,
        "error_code": result.error_code,
    }
async def update_node(self, cmd: "UpdateNodeCommand") -> dict:
    if cmd.operator_id:
        allowed = await self._check_node_permission(cmd.node_id, cmd.operator_id, "write")
        self._require_permission(allowed, cmd.node_id, "write")
    result = await self._service.update_node(cmd)
    return {
        "success": result.success,
        "node_id": result.node_id,
        "status": result.status,
        "reply": result.message,
        "error_code": result.error_code,
    }
async def discard_node(self, cmd: "DiscardNodeCommand") -> dict:
    if cmd.operator_id:
        allowed = await self._check_node_permission(cmd.node_id, cmd.operator_id, "delete")
        self._require_permission(allowed, cmd.node_id, "delete")
    result = await self._service.discard_node(cmd)
```

```
        return {
            "success": result.success,
            "node_id": result.node_id,
            "reply": result.message,
            "error_code": result.error_code,
        }
    }

    async def activate_node(self, cmd: "ActivateNodeCommand") -> dict:
        try:
            result = await self._service.activate_node(cmd)
            return {
                "success": result.success,
                "node_id": result.node_id,
                "status": result.status,
                "reply": result.message,
                "error_code": result.error_code,
            }
        except Exception as e:
            logger.error("activate_node failed: %s", e)
            return {"success": False, "node_id": cmd.node_id, "reply": f"节点激活失败: {e}"}

    async def create_deliverable(self, cmd: "CreateDeliverableCommand") -> dict:
        result = await self._service.create_deliverable(cmd)
        return {
            "success": result.success,
            "node_id": result.node_id,
            "reply": result.message,
            "error_code": result.error_code,
        }

    async def update_deliverable_progress(self, cmd: "UpdateDeliverableProgressCommand") -> dict:
        result = await self._service.update_deliverable_progress(cmd)
        return {
            "success": result.success,
            "node_id": result.node_id,
            "status": result.status,
            "progress": result.progress,
            "reply": result.message,
            "affected_ancestors": result.affected_downstream,
            "error_code": result.error_code,
        }

    async def add_dependency(self, cmd: "AddDependencyCommand") -> dict:
        result = await self._service.add_dependency(cmd)
        return {
            "success": result.success,
            "node_id": result.node_id,
            "reply": result.message,
```

```

        "error_code": result.error_code,
    }
    async def remove_dependency(self, cmd: "RemoveDependencyCommand") -> dict:
        result = await self._service.remove_dependency(cmd)
        return {
            "success": result.success,
            "node_id": result.node_id,
            "reply": result.message,
            "error_code": result.error_code,
        }
    async def mount_child(self, cmd: "MountChildCommand") -> dict:
        result = await self._service.mount_child(cmd)
        return {
            "success": result.success,
            "node_id": result.node_id,
            "reply": result.message,
            "error_code": result.error_code,
        }
    async def unmount_child(self, cmd: "UnmountChildCommand") -> dict:
        result = await self._service.unmount_child(cmd)
        return {
            "success": result.success,
            "node_id": result.node_id,
            "reply": result.message,
            "error_code": result.error_code,
        }
    async def get_node_detail(self, node_id: str) -> dict:
        detail = await self._service.get_node_detail(node_id)
        if detail is None:
            return {"success": False, "reply": f"节点 {node_id} 不存在"}
        return {"success": True, "data": detail, "reply": f"节点「{detail['node_name']}」查询成功"}

```

文件编号: 15 文件路径: emily-core/emily_core/application/permission_app.py

```

from __future__ import annotations
import logging
from typing import Optional
from emily_core.services.permission_service import PermissionService
logger = logging.getLogger("emily.permission.app")
class PermissionApplication:
    def __init__(self, service: PermissionService):
        self._service = service
    async def check_permission(self, user_id: str, sop_id: str) -> dict:
        try:
            result = await self._service.check(user_id, sop_id)

```

```

signature = json.dumps(
    {"params": bft.parameters, "returns": "dict"},
    ensure_ascii=False,
)
ok = ToolRegistryRepo.upsert(
    api_id=bft.name,
    signature=signature,
    display_name=(
        bft.description.split("。")[0][:80]
        if bft.description
        else f"Tool: {bft.name}"
    ),
    category=category,
    permission_flag=perm_flag,
    handler_module=getattr(bft.handler, "__module__", module_path),
)
if ok:
    print(f" [DB] Tool '{bft.name}' registered in tool_registry table")
else:
    print(f" [DB] Tool '{bft.name}' DB upsert FAILED")
    return False
return True

def cmd_list():
    _init_db()
    from emily_core.repositories.tool_registry_repo import ToolRegistryRepo
    rows = ToolRegistryRepo.get_all()
    if not rows:
        print("(no registered APIs)")
        return
    print(f"/n{'API ID':<24s} {'Category':<10s} {'Active':<6s} Description")
    print("-" * 80)
    for r in rows:
        active = "YES" if r.get("is_active") else "NO"
        print(
            f" {r['api_id']:<24s} {r.get('category', ''):<10s} "
            f"{active:<6s} {r.get('display_name', '')}"
        )

def cmd_help_api(api_id: str):
    _init_db()
    from emily_core.repositories.tool_registry_repo import ToolRegistryRepo
    rows = ToolRegistryRepo.get_all()
    found = None
    for r in rows:
        if r["api_id"] == api_id:

```

```
        found = r
        break
    if found:
        print(f"/nAPI: {found['api_id']}")
        print(f" 描述: {found.get('display_name', '')}")
        print(f" 类别: {found.get('category', '')}")
        print(f" 权限: {found.get('permission_flag', '')}")
        print(f" 模块: {found.get('handler_module', '')}")
        try:
            sig = json.loads(found.get("signature", "{}"))
            print(f" 签名: {json.dumps(sig, ensure_ascii=False, indent=2)}")
        except json.JSONDecodeError:
            print(f" 签名: {found.get('signature', '')}")
    else:
        print(f"[ERROR] API '{api_id}' not found in tool_registry")
def main():
    parser = argparse.ArgumentParser(description="API 注册器")
    parser.add_argument("--api", help="注册单个 API")
    parser.add_argument("--all", action="store_true", help="注册全部 API")
    parser.add_argument("--list", action="store_true", help="查看已注册 API")
    parser.add_argument("--help-api", help="查看某个 API 帮助")
    args = parser.parse_args()
    if args.list:
        cmd_list()
    elif args.help_api:
        cmd_help_api(args.help_api)
    elif args.all:
        success = 0
        for e in _TOOL_SCRIPTS:
            api_id = e[0]
            print(f"/n=== Registering {api_id} ===")
            if do_register(api_id):
                success += 1
        print(f"/nDone: {success}/{len(_TOOL_SCRIPTS)} APIs registered")
    elif args.api:
        do_register(args.api)
    else:
        parser.print_help()
if __name__ == "__main__":
    main()
```

文件编号: 205 文件路径: scripts/rescan_files.py

```
from __future__ import annotations
import argparse
```



```
import hashlib
import logging
import os
import uuid
from datetime import datetime, timezone
from pathlib import Path
from sqlalchemy import Boolean, Column, create_engine, Integer, String, Text
from sqlalchemy.orm import DeclarativeBase, Session, sessionmaker
PROJECT_ROOT = Path(__file__).resolve().parent.parent
DEFAULT_SCAN_DIR = PROJECT_ROOT / "emily-data" / "files"
DEFAULT_ATTACHMENTS_DIR = PROJECT_ROOT / "emily-data" / "attachments"
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)
logger = logging.getLogger("rescan_files")
class Base(DeclarativeBase):
    pass
class File(Base):
    __tablename__ = "files"
    id = Column(String, primary_key=True)
    file_no = Column(String(50), unique=True, nullable=False)
    project_id = Column(String, nullable=True)
    message_id = Column(String, nullable=True)
    filename = Column(String(500), nullable=False)
    file_type = Column(String(100))
    bucket = Column(String(100))
    object_key = Column(String)
    storage_path = Column(String)
    file_size = Column(Integer)
    uploaded_by = Column(String, nullable=True)
    parse_status = Column(String(50), default="pending")
    created_at = Column(String)
    file_ext = Column(String(50), default="")
    file_url = Column(String(1000), default="")
    file_hash = Column(String(256), default="")
    version = Column(String(50), default="V1.0")
    is_latest = Column(Boolean, default=True)
    parent_file_id = Column(String, nullable=True)
    confidentiality = Column(Integer, default=0)
    creator_id = Column(String, nullable=True)
    updated_at = Column(String)
    is_deleted = Column(Boolean, default=False)
```

```

source_module_id = Column(String(100), default="")
source_module_type = Column(String(50), default="")
_EXT_TYPE_MAP: dict[str, str] = {
    ".pdf": "pdf",
    ".doc": "doc", ".docx": "doc",
    ".xls": "xls", ".xlsx": "xls",
    ".ppt": "ppt", ".pptx": "ppt",
    ".jpg": "image", ".jpeg": "image", ".png": "image",
    ".gif": "image", ".bmp": "image", ".webp": "image",
    ".mp3": "audio", ".wav": "audio", ".amr": "audio",
    ".mp4": "video", ".avi": "video", ".mkv": "video",
    ".zip": "archive", ".rar": "archive", ".7z": "archive",
    ".txt": "text", ".md": "text", ".csv": "csv",
    ".json": "data", ".xml": "data",
}
def _utc_now() -> str:
    return datetime.now(timezone.utc).isoformat()
def compute_sha256(file_path: Path) -> str:
    h = hashlib.sha256()
    with open(file_path, "rb") as f:
        while True:
            chunk = f.read(8192)
            if not chunk:
                break
            h.update(chunk)
    return h.hexdigest()
def infer_file_type(ext: str) -> str:
    return _EXT_TYPE_MAP.get(ext.lower(), "file")
def generate_file_no(session: Session) -> str:
    today_str = datetime.now(timezone.utc).strftime("%Y%m%d")
    prefix = f"FIL-{today_str}-"
    last = (
        session.query(File)
        .filter(File.file_no.like(f"{prefix}%"))
        .order_by(File.file_no.desc())
        .first()
    )
    if last is None:
        return f"{prefix}0001"
    seq_str = last.file_no[len(prefix):]
    try:
        seq = int(seq_str) + 1
    except ValueError:
        seq = 1

```

```
    return f"{prefix}{{seq:04d}}"
def scan_directory(scan_dir: Path) -> list[Path]:
    if not scan_dir.exists():
        logger.warning(" 扫描目录不存在: %s", scan_dir)
        return []
    files = []
    for root, _dirs, filenames in os.walk(scan_dir):
        for fn in filenames:
            fp = Path(root) / fn
            if fn.startswith(".") or fn == ".gitkeep":
                continue
            files.append(fp)
    return files
def find_existing_record(session: Session, storage_path: str, filename: str, file_size: int):
    if storage_path:
        record = session.query(File).filter(
            File.storage_path == storage_path,
            File.is_deleted == False, # noqa: E712
        ).first()
        if record:
            return record
    record = session.query(File).filter(
        File.filename == filename,
        File.file_size == file_size,
        File.is_deleted == False, # noqa: E712
    ).first()
    if record:
        return record
    return None
def insert_file_record(session: Session, *, file_path: Path, scan_root: Path) -> str:
    file_no = generate_file_no(session)
    ext = file_path.suffix.lower()
    file_size = file_path.stat().st_size
    file_hash = compute_sha256(file_path)
    rel_path = str(file_path.relative_to(scan_root)).replace("//", "/")
    file_type = infer_file_type(ext)
    now_iso = _utc_now()
    new_id = str(uuid.uuid4())
    record = File(
        id=new_id,
        file_no=file_no,
        filename=file_path.name,
        project_id=None,
        message_id=None,
```

```
        file_type=file_type,
        storage_path=rel_path,
        file_size=file_size,
        uploaded_by=None,
        parse_status="pending",
        created_at=now_iso,
        file_ext=ext.lstrip("."),
        file_url=rel_path,
        file_hash=file_hash,
        version="V1.0",
        is_latest=True,
        parent_file_id=None,
        confidentiality=0,
        creator_id=None,
        updated_at=now_iso,
        is_deleted=False,
        source_module_id="",
        source_module_type="",
    )
    session.add(record)
    session.flush()
    return file_no
def run_rescan(
    scan_dirs: list[Path],
    db_url: str,
    dry_run: bool = False,
) -> None:
    engine = create_engine(db_url, echo=False, pool_pre_ping=True)
    SessionLocal = sessionmaker(
        autocommit=False, autoflush=False, bind=engine, expire_on_commit=False,
    )
    total_scanned = 0
    total_existing = 0
    total_inserted = 0
    total_error = 0
    inserted_details: list[dict] = []
    existing_details: list[dict] = []
    error_details: list[str] = []
    for scan_dir in scan_dirs:
        logger.info(" 扫描目录: %s", scan_dir)
        file_paths = scan_directory(scan_dir)
        logger.info(" 发现 %d 个文件", len(file_paths))
        for fp in file_paths:
            total_scanned += 1
```

```
rel_path = str(fp.relative_to(scan_dir)).replace("//", "/")
file_size = fp.stat().st_size
try:
    session = SessionLocal()
    try:
        existing = find_existing_record(session, rel_path, fp.name, file_size)
        if existing:
            total_existing += 1
            existing_details.append({
                "file": str(fp),
                "file_no": existing.file_no,
                "matched_by": "storage_path" if existing.storage_path == rel_path else "filename+size"
            })
            logger.debug(" 已存在: %s (file_no=%s)", fp.name, existing.file_no)
        else:
            if dry_run:
                total_inserted += 1
                inserted_details.append({
                    "file": str(fp),
                    "file_no": "[DRY-RUN]",
                    "rel_path": rel_path,
                    "size": file_size,
                })
                logger.info("[DRY-RUN] 将补录: %s (%d bytes)", fp.name, file_size)
            else:
                file_no = insert_file_record(
                    session,
                    file_path=fp,
                    scan_root=scan_dir,
                )
                session.commit()
                total_inserted += 1
                inserted_details.append({
                    "file": str(fp),
                    "file_no": file_no,
                    "rel_path": rel_path,
                    "size": file_size,
                })
                logger.info(" 已补录: %s → %s", fp.name, file_no)
    finally:
        session.close()
except Exception as e:
    total_error += 1
    error_details.append(f"{fp}: {e}")
```

```
        logger.error(" 处理失败: %s — %s", fp, e)
    print("/n" + "=" * 60)
    print(" 扫描补录报告")
    print("=" * 60)
    print(f" 扫描目录 : {' , '.join(str(d) for d in scan_dirs)}")
    print(f" 扫描文件 : {total_scanned}")
    print(f" 已存在   : {total_existing}")
    print(f" 新补录   : {total_inserted}" + (" (DRY-RUN)" if dry_run else ""))
    print(f" 失败     : {total_error}")
    print("-" * 60)
    if existing_details:
        print("/n 已存在文件 (跳过): ")
        for item in existing_details:
            print(f" {item['file']} -> {item['file_no']} (匹配: {item['matched_by']})")
    if inserted_details:
        print("/n 新补录文件: ")
        for item in inserted_details:
            size_kb = item['size'] / 1024
            print(f" {item['file']} -> {item['file_no']} ({size_kb:.1f} KB) path={item['rel_path']}")
    if error_details:
        print("/n 失败文件: ")
        for msg in error_details:
            print(f" {msg}")
    print("=" * 60)
    engine.dispose()
def main():
    parser = argparse.ArgumentParser(
        description=" 扫描本地文件目录, 对照 files 表补录缺失记录",
    )
    parser.add_argument(
        "--dir",
        type=str,
        default=str(DEFAULT_SCAN_DIR),
        help=f" 扫描目录 (默认: {DEFAULT_SCAN_DIR})",
    )
    parser.add_argument(
        "--include-attachments",
        action="store_true",
        help=" 同时扫描 emily-data/attachments 目录",
    )
    parser.add_argument(
        "--db-url",
        type=str,
        default="postgresql://emily:emily_secret_2026@localhost:25432/emily",
    )
```

```
    help="PostgreSQL 连接 URL (默认: postgresql://emily:emily_secret_2026@localhost:25432/emily)",
)
parser.add_argument(
    "--dry-run",
    action="store_true",
    help="预览模式: 只输出报告, 不写入数据库",
)
args = parser.parse_args()
scan_dirs = [Path(args.dir)]
if args.include_attachments:
    scan_dirs.append(DEFAULT_ATTACHMENTS_DIR)
logger.info("数据库: %s", args.db_url.replace("emily_secret_2026", "****"))
run_rescan(scan_dirs, db_url=args.db_url, dry_run=args.dry_run)
if __name__ == "__main__":
    main()
```

文件编号: 206 文件路径: scripts/self_check.py

```
from __future__ import annotations
import argparse
import io
import json
import logging
import os
import subprocess
import sys
from datetime import datetime, timezone, timedelta
from pathlib import Path
_HERE = Path(__file__).resolve().parent
_CORE_DIR = _HERE.parent / "emily-core"
if str(_CORE_DIR) not in sys.path:
    sys.path.insert(0, str(_CORE_DIR))
logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s] %(name)s: %(message)s")
logger = logging.getLogger("self_check")
BEIJING_TZ = timezone(timedelta(hours=8))
def _init_db(db_url: str = "") -> None:
    from emily_core.infrastructure.database.session import init_db
    if db_url:
        init_db(db_url=db_url)
    else:
        db_url_env = os.environ.get("EMILY_DATABASE_URL", "")
        if db_url_env:
            init_db(db_url=db_url_env)
        else:
            pg_port = int(os.environ.get("EMILY_PG_PORT", "")) if os.environ.get("EMILY_PG_PORT") else None
```

```

    if not pg_port:
        try:
            r = subprocess.run(["docker", "port", "emily-postgres", "5432/tcp"], capture_output=True, text=True)
            pg_port = int(r.stdout.strip().rsplit(":", 1)[-1]) if r.returncode == 0 and r.stdout.strip() else None
        except Exception:
            pg_port = 5432
    init_db(pg_host=os.environ.get("EMILY_PG_HOST", "127.0.0.1"), pg_port=pg_port,
           pg_db=os.environ.get("EMILY_PG_DB", "emily"),
           pg_user=os.environ.get("EMILY_PG_USER", "emily"),
           pg_password=os.environ.get("EMILY_PG_PASSWORD", "emily_secret_2026"))

def self_check(*, db_url: str = "", dry_run: bool = False) -> dict:
    _init_db(db_url)
    from emily_core.infrastructure.database.session import get_session
    from emily_core.infrastructure.database.models import User, Project, Event, Task, ProjectNode, ProjectWorldBook
    result = {
        "checked_at": datetime.now(BEIJING_TZ).isoformat(),
        "dry_run": dry_run,
    }
    with get_session() as session:
        total_users = session.query(User).filter(User.is_deleted == False).count()
        active_users = session.query(User).filter(User.is_deleted == False, User.status == "active").count()
        admin_users = session.query(User).filter(User.is_deleted == False, User.is_admin == True).count()
        result["users"] = {"total": total_users, "active": active_users, "admins": admin_users}
        total_projects = session.query(Project).filter(Project.is_deleted == False).count()
        active_projects = session.query(Project).filter(Project.is_deleted == False, Project.status == "active").count()
        result["projects"] = {"total": total_projects, "active": active_projects}
        event_count = session.query(Event).count()
        task_count = session.query(Task).count()
        node_count = session.query(ProjectNode).filter(ProjectNode.is_discarded == False).count()
        result["business"] = {"events": event_count, "tasks": task_count, "nodes": node_count}
        wb_count = session.query(ProjectWorldBook).count()
        wb_activated = session.query(ProjectWorldBook).filter(ProjectWorldBook.is_activated == True).count()
        result["world_books"] = {"total": wb_count, "activated": wb_activated}
        sop_count = 0
    try:
        from emily_core.skill.registry import SkillRegistry
        skill_dir = "/app/skills"
        if not Path(skill_dir).exists():
            dev_dir = str(Path(__file__).resolve().parent.parent / "emily-data" / "skills")
            if Path(dev_dir).exists():
                skill_dir = dev_dir
        if skill_dir and Path(skill_dir).exists():
            reg = SkillRegistry(skill_directory=skill_dir)
            reg.load()

```



```

        sop_count = len(reg.list_sop_ids())
    except Exception:
        pass
    result["knowledge"] = {"sop_count": sop_count}
    return result
def _format_self_check(result: dict) -> str:
    lines = []
    lines.append("Emily 系统自检报告")
    lines.append("=" * 40)
    lines.append(f" 检查时间: {result['checked_at']}")
    lines.append("=" * 40)
    u = result.get("users", {})
    lines.append(f"/n 用户: {u.get('active', 0)} 活跃 / {u.get('total', 0)} 总计 / {u.get('admins', 0)} 管理")
    p = result.get("projects", {})
    lines.append(f" 项目: {p.get('active', 0)} 活跃 / {p.get('total', 0)} 总计")
    b = result.get("business", {})
    lines.append(f" 业务: {b.get('events', 0)} 事件 / {b.get('tasks', 0)} 任务 / {b.get('nodes', 0)} 节点")
    wb = result.get("world_books", {})
    lines.append(f" 世界书: {wb.get('total', 0)} 份 / {wb.get('activated', 0)} 已激活")
    k = result.get("knowledge", {})
    lines.append(f" 知识库: {k.get('sop_count', 0)} 个 SOP")
    return "/n".join(lines)
def main():
    sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8', errors='replace')
    sys.stderr = io.TextIOWrapper(sys.stderr.buffer, encoding='utf-8', errors='replace')
    parser = argparse.ArgumentParser(description="Emily 系统自检")
    parser.add_argument("--db-url", default="")
    parser.add_argument("--dry-run", action="store_true")
    parser.add_argument("--json", action="store_true")
    args = parser.parse_args()
    result = self_check(db_url=args.db_url, dry_run=args.dry_run)
    if args.json:
        print(json.dumps(result, ensure_ascii=False, indent=2))
    else:
        print(_format_self_check(result))
if __name__ == "__main__":
    main()

```

文件编号: 207 文件路径: scripts/sop_to_skill.py

```

import argparse
import os
import sys
from pathlib import Path
PROJECT_ROOT = Path(__file__).resolve().parents[1]

```

```
sys.path.insert(0, str(PROJECT_ROOT / "emily-core"))
def _find_sop_dir() -> Path:
    candidates = [
        PROJECT_ROOT / "emily-data" / "sops",
        Path("/app/sops"),
    ]
    for d in candidates:
        if d.exists():
            return d
    raise FileNotFoundError("SOP 目录未找到")
def _find_skill_dir() -> Path:
    candidates = [
        PROJECT_ROOT / "emily-data" / "skills",
        Path("/app/skills"),
    ]
    for d in candidates:
        if d.exists():
            return d
    default = candidates[0]
    default.mkdir(parents=True, exist_ok=True)
    return default
def _load_sop(sop_id: str, sop_dir: Path) -> str:
    for f in sop_dir.glob(f"*{sop_id}*.md"):
        return f.read_text(encoding="utf-8")
    raise FileNotFoundError(f"SOP 文件未找到: {sop_id} (在 {sop_dir})")
def _call_llm(sop_text: str, api_key: str, base_url: str, model: str) -> str:
    from openai import OpenAI
    client = OpenAI(api_key=api_key, base_url=base_url)
    prompt_path = PROJECT_ROOT / "emily-data" / "prompts" / "sop_to_skill.md"
    if prompt_path.exists():
        system_prompt = prompt_path.read_text(encoding="utf-8")
    else:
        system_prompt = " 将以下 SOP 文档转换为 Skill YAML (三段结构: instructions / tools / steps), 不要"
    system_prompt = system_prompt.replace("{sop_text}", sop_text)
    response = client.chat.completions.create(
        model=model,
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": " 请转换此 SOP 文档为 Skill YAML。"},
        ],
        temperature=0,
    )
    content = response.choices[0].message.content or ""
    if "`yaml" in content:
```

```
        content = content.split("```yaml")[1].split("```")[0]
    elif "```" in content:
        content = content.split("```")[1].split("```")[0]
    return content.strip()
def _validate_yaml(text: str) -> bool:
    try:
        import yaml
        data = yaml.safe_load(text)
        return isinstance(data, dict) and "skill_id" in data and "steps" in data
    except Exception:
        return False
def convert_sop(sop_id: str, dry_run: bool = False) -> str | None:
    sop_dir = _find_sop_dir()
    skill_dir = _find_skill_dir()
    sop_text = _load_sop(sop_id, sop_dir)
    print(f" 加载 SOP: {sop_id} ({len(sop_text)} chars)")
    api_key = os.environ.get("DEEPSEEK_API_KEY", "")
    base_url = os.environ.get("DEEPSEEK_BASE_URL", "https://api.deepseek.com")
    model = os.environ.get("DEEPSEEK_MODEL", "deepseek-chat")
    if not api_key:
        print(" 错误: 未设置 DEEPSEEK_API_KEY 环境变量")
        return None
    yaml_text = _call_llm(sop_text, api_key, base_url, model)
    if not _validate_yaml(yaml_text):
        print(" 警告: LLM 输出不是合法 Skill YAML, 请人工审校")
    if dry_run:
        print("/n" + "=" * 60)
        print(yaml_text)
        print("=" * 60)
        return yaml_text
    import yaml
    data = yaml.safe_load(yaml_text)
    skill_id = data.get("skill_id", sop_id)
    output_path = skill_dir / f"{skill_id}.skill.yaml"
    output_path.write_text(yaml_text, encoding="utf-8")
    print(f" 写入: {output_path}")
    return yaml_text
def _notify_reload() -> bool:
    import json
    try:
        import urllib.request
        core_host = os.environ.get("EMILY_CORE_HOST", "localhost")
        core_port = os.environ.get("EMILY_CORE_PORT", "18080")
        url = f"http://{core_host}:{core_port}/api/v1/skills/reload"
```

```

req = urllib.request.Request(url, method="POST", data=b"")
with urllib.request.urlopen(req, timeout=5) as resp:
    result = json.loads(resp.read().decode("utf-8"))
    if result.get("ok"):
        print(f"✓ EmilyCore 热重载成功: {result.get('total', 0)} skill(s)")
        return True
    else:
        print(f"✗ EmilyCore 热重载失败: {result.get('error', '未知错误')}")
        return False
except Exception as e:
    print(f"△ 无法通知 EmilyCore 热重载 (容器未运行?): {e}")
    return False
def main():
    parser = argparse.ArgumentParser(description="SOP-to-Skill 转换器")
    parser.add_argument("--sop", help="SOP 编号 (如 SOP-002-REC) ")
    parser.add_argument("--all", action="store_true", help=" 批量转换全部 SOP")
    parser.add_argument("--dry-run", action="store_true", help=" 输出到 stdout 不写文件")
    parser.add_argument("--notify", action="store_true", help=" 转换完成后通知 EmilyCore 热重载")
    args = parser.parse_args()
    if not args.sop and not args.all:
        parser.print_help()
        return
    converted = 0
    if args.all:
        sop_dir = _find_sop_dir()
        for f in sorted(sop_dir.glob("SOP-*.md")):
            sop_id = f.stem.split("-")[0] + "-" + f.stem.split("-")[1] + "-" + f.stem.split("-")[2]
            print(f"/n 转换: {sop_id}")
            result = convert_sop(sop_id, dry_run=args.dry_run)
            if result:
                converted += 1
    else:
        result = convert_sop(args.sop, dry_run=args.dry_run)
        if result:
            converted += 1
    if args.notify and converted > 0 and not args.dry_run:
        print(f"/n 已转换 {converted} 个 Skill, 通知 EmilyCore 热重载...")
        _notify_reload()
if __name__ == "__main__":
    main()

```

文件编号: 208 文件路径: scripts/update_world_book.py

```

from __future__ import annotations
import argparse

```

```

import asyncio
import io
import json
import logging
import os
import subprocess
import sys
from pathlib import Path
_HERE = Path(__file__).resolve().parent
_CORE_DIR = _HERE.parent / "emily-core"
if str(_CORE_DIR) not in sys.path:
    sys.path.insert(0, str(_CORE_DIR))
logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s] %(name)s: %(message)s")
logger = logging.getLogger("update_world_book")
def _init_db(db_url: str = "") -> None:
    from emily_core.infrastructure.database.session import init_db
    if db_url:
        init_db(db_url=db_url)
    else:
        db_url_env = os.environ.get("EMILY_DATABASE_URL", "")
        if db_url_env:
            init_db(db_url=db_url_env)
        else:
            pg_port = int(os.environ.get("EMILY_PG_PORT", "")) if os.environ.get("EMILY_PG_PORT") else None
            if not pg_port:
                try:
                    r = subprocess.run(["docker", "port", "emily-postgres", "5432/tcp"], capture_output=True, text=True)
                    pg_port = int(r.stdout.strip().rsplit(":", 1)[-1]) if r.returncode == 0 and r.stdout.strip() else None
                except Exception:
                    pg_port = 5432
            init_db(pg_host=os.environ.get("EMILY_PG_HOST", "127.0.0.1"), pg_port=pg_port,
                    pg_db=os.environ.get("EMILY_PG_DB", "emily"),
                    pg_user=os.environ.get("EMILY_PG_USER", "emily"),
                    pg_password=os.environ.get("EMILY_PG_PASSWORD", "emily_secret_2026"))
async def update_world_book(project_id: str, *, db_url: str = "", dry_run: bool = False) -> dict:
    _init_db(db_url)
    from emily_core.services.world_book_service import ProjectWorldBookService
    service = ProjectWorldBookService()
    return await service.update_stale(project_id, dry_run=dry_run)
def main():
    sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8', errors='replace')
    sys.stderr = io.TextIOWrapper(sys.stderr.buffer, encoding='utf-8', errors='replace')
    parser = argparse.ArgumentParser(description=" 增量更新世界书")
    parser.add_argument("--project-id", help=" 项目 ID")

```

```

parser.add_argument("--all", action="store_true")
parser.add_argument("--db-url", default="")
parser.add_argument("--dry-run", action="store_true")
args = parser.parse_args()
if not args.project_id and not args.all:
    parser.error(" 请指定 --project-id 或 --all")
if args.all:
    _init_db(args.db_url)
    from emily_core.services.world_book_service import ProjectWorldBookService
    service = ProjectWorldBookService()
    results = asyncio.run(service.update_all(dry_run=args.dry_run))
    for r in results:
        print(json.dumps({k: v for k, v in r.items() if k not in ("content_json", "drift_details")}, ensure_ascii=False))
    else:
        result = asyncio.run(update_world_book(args.project_id, db_url=args.db_url, dry_run=args.dry_run))
        print(json.dumps({k: v for k, v in result.items() if k not in ("content_json",)}, ensure_ascii=False, indent=4))
if __name__ == "__main__":
    main()

```

文件编号：209 文件路径：data/plugins/emily_agent/_conf_schema.json

```

{
  "emycore_url": {
    "description": "Emily Core 容器 API 地址",
    "type": "string",
    "hint": " 独立业务核心容器的 HTTP 地址 (Docker 内网用服务名) ",
    "default": "http://emily-core:18080"
  },
  "emycore_sse_url": {
    "description": "Emily Core SSE 出站推送地址",
    "type": "string",
    "hint": " 为空时自动取 {emycore_url}/api/v1/events/outbound",
    "default": ""
  },
  "emycore_api_token": {
    "description": "Emily Core API 鉴权令牌",
    "type": "string",
    "hint": " 与 Core 容器 EMILY_API_TOKEN 一致；为空时不做鉴权（仅内网部署）",
    "default": ""
  }
}

```

文件编号：210 文件路径：data/plugins/emily_agent/adapters/api_client.py

```

from __future__ import annotations
import logging

```

```
from typing import TYPE_CHECKING
import aiohttp
if TYPE_CHECKING:
    from .standard.message import StandardMessage
    from .standard.reply import ReplyMessage
logger = logging.getLogger("emily.plugin.api_client")
class EmilyApiClient:
    def __init__(
        self,
        base_url: str = "http://emily-core:18080",
        api_token: str = "",
        timeout: float = 30.0,
    ):
        self.base_url = base_url.rstrip("/")
        self.api_token = api_token
        self.timeout = aiohttp.ClientTimeout(total=timeout)
        self._session: aiohttp.ClientSession | None = None
    async def _ensure_session(self) -> aiohttp.ClientSession:
        if self._session is None or self._session.closed:
            headers = {}
            if self.api_token:
                headers["X-Emily-Token"] = self.api_token
            self._session = aiohttp.ClientSession(timeout=self.timeout, headers=headers)
        return self._session
    async def close(self) -> None:
        if self._session and not self._session.closed:
            await self._session.close()
    def get_sse_url(self) -> str:
        return f"{self.base_url}/api/v1/events/outbound"
    async def send_message(self, msg: "StandardMessage") -> "ReplyMessage | None":
        from .standard.reply import ReplyMessage
        session = await self._ensure_session()
        url = f"{self.base_url}/api/v1/message/send"
        payload = self._message_to_dict(msg)
        try:
            async with session.post(url, json=payload) as resp:
                if resp.status == 204:
                    return None
                if resp.status != 200:
                    text = await resp.text()
                    logger.warning("send_message %d: %s", resp.status, text[:200])
                    return None
                data = await resp.json()
                return ReplyMessage(
```

```
        conversation_id=data.get("conversation_id", ""),
        content=data.get("content", ""),
        reply_to_message_id=data.get("reply_to_message_id"),
    )
except Exception as e:
    logger.error("send_message failed: %s", e)
    return None
async def terminate_session(self, conversation_id: str) -> bool:
    session = await self._ensure_session()
    url = f"{self.base_url}/api/v1/session/terminate"
    try:
        async with session.post(url, json={"conversation_id": conversation_id}) as resp:
            if resp.status != 200:
                return False
            data = await resp.json()
            return bool(data.get("terminated"))
    except Exception as e:
        logger.error("terminate_session failed: %s", e)
        return False
async def health_check(self) -> dict:
    session = await self._ensure_session()
    url = f"{self.base_url}/api/v1/health"
    try:
        async with session.get(url) as resp:
            if resp.status == 200:
                return await resp.json()
            return {"status": "error", "code": resp.status}
    except Exception as e:
        logger.error("health_check failed: %s", e)
        return {"status": "unreachable", "error": str(e)}
@staticmethod
def _message_to_dict(msg: "StandardMessage") -> dict:
    return {
        "message_id": msg.message_id,
        "platform": msg.platform,
        "conversation_type": msg.conversation_type,
        "conversation_id": msg.conversation_id,
        "sender_id": msg.sender_id,
        "sender_name": msg.sender_name,
        "group_id": msg.group_id,
        "group_name": msg.group_name,
        "content": msg.content,
        "is_at_bot": msg.is_at_bot,
        "mentioned_user_ids": list(msg.mentioned_user_ids or []),
    }
```



```
    "reply_to_message_id": msg.reply_to_message_id,
    "msg_type": getattr(msg, "msg_type", 1),
    "attachments": list(getattr(msg, "attachments", []) or []),
}
```

文件编号：211 文件路径：data/plugins/emily_agent/adapters/astrbot/inbound_adapter.py

```
import logging
from typing import TYPE_CHECKING
from ..standard.message import StandardMessage
if TYPE_CHECKING:
    from astrbot.core.platform.astr_message_event import AstrMessageEvent
logger = logging.getLogger("emily.adapter.inbound")
class AstrBotInboundAdapter:
    @staticmethod
    def to_standard_message(event: "AstrMessageEvent") -> StandardMessage:
        from astrbot.core.platform.message_type import MessageType
        msg_type = event.get_message_type()
        if msg_type == MessageType.GROUP_MESSAGE:
            conversation_type = "group"
        elif msg_type == MessageType.PRIVATE_MESSAGE:
            conversation_type = "private"
        else:
            conversation_type = "unknown"
            logger.warning("Unknown message type: %s, treating as unknown", msg_type)
        sender_id = event.get_sender_id()
        sender_name = event.get_sender_name()
        group_id = None
        if conversation_type == "group":
            gid = event.get_group_id()
            if gid:
                group_id = gid
        content = event.message_str or ""
        msg_type = 1
        attachments: list[dict] = []
        receiver_id = ""
        self_id = event.get_self_id()
        if conversation_type == "group":
            receiver_id = self_id or ""
        messages = event.get_messages()
        for comp in messages:
            if hasattr(comp, "qq"):
                uid = str(comp.qq)
                mentioned_ids.append(uid)
                if uid == self_id:
```

```
        is_at_bot = True
    if hasattr(comp, "type") and getattr(comp, "type", None) == "at_all":
        mentioned_ids.append("all")
    comp_type = getattr(comp, "type", None)
    comp_data = getattr(comp, "data", None) or {}
    if comp_type == "image" or (comp_data and comp_data.get("url")):
        if msg_type == 1:
            msg_type = 2
        url = comp_data.get("url") or getattr(comp, "url", "")
        attachments.append({
            "type": 2,
            "url": url,
            "file_name": comp_data.get("file", "") or comp_data.get("summary", ""),
            "file_size": comp_data.get("file_size", 0),
            "summary": comp_data.get("summary", ""),
        })
    elif comp_type == "record" or (comp_data and "time" in comp_data):
        if msg_type == 1:
            msg_type = 4
        url = comp_data.get("url") or getattr(comp, "url", "")
        attachments.append({
            "type": 4,
            "url": url,
            "file_name": comp_data.get("file", ""),
            "file_size": comp_data.get("file_size", 0),
        })
    elif comp_type == "video" or (comp_data and comp_data.get("thumb")):
        msg_type = 5
        url = comp_data.get("url") or getattr(comp, "url", "")
        attachments.append({
            "type": 5,
            "url": url,
            "file_name": comp_data.get("file", ""),
            "file_size": comp_data.get("file_size", 0),
            "thumb": comp_data.get("thumb", ""),
        })
    elif comp_type == "file" or (comp_data and comp_data.get("name")):
        if msg_type == 1:
            msg_type = 3
        url = comp_data.get("url") or getattr(comp, "url", "")
        attachments.append({
            "type": 3,
            "url": url,
            "file_name": comp_data.get("name", ""),
        })
```

```

        "file_size": comp_data.get("size", 0),
    })
    message_id = ""
    msg_obj = event.message_obj
    if msg_obj:
        msg_id_attr = getattr(msg_obj, "message_id", None)
        if msg_id_attr:
            message_id = str(msg_id_attr)
    reply_to = None
    if msg_obj:
        raw = getattr(msg_obj, "raw_message", None)
        if isinstance(raw, dict):
            reply_to = str(raw.get("reply_message_id", "")) or None
    standard = StandardMessage(
        message_id=message_id,
        platform=event.get_platform_name() or "napcat",
        conversation_type=conversation_type,
        conversation_id=group_id or sender_id,
        sender_id=sender_id,
        sender_name=sender_name,
        group_id=group_id,
        content=content,
        is_at_bot=is_at_bot,
        mentioned_user_ids=mentioned_ids,
        reply_to_message_id=reply_to,
        msg_type=msg_type,
        attachments=attachments,
    )
    logger.debug(
        "inbound: type=%s, sender=%s, at_bot=%s, content_preview=%.50s",
        conversation_type, sender_name, is_at_bot, content,
    )
    return standard

```

文件编号：212 文件路径：data/plugins/emily_agent/adapters/astrobot/outbound_sender.py

```

import asyncio
import logging
from typing import TYPE_CHECKING
from ..standard.reply import ReplyMessage
if TYPE_CHECKING:
    from astrobot.core.platform.astr_message_event import AstrMessageEvent
logger = logging.getLogger("emily.adapter.outbound")
class AstrBotOutboundSender:
    @staticmethod

```

```
async def send(reply: ReplyMessage, event: "AstrMessageEvent") -> None:
    file_paths = getattr(reply, "file_paths", None) or []
    if file_paths:
        result = AstrBotOutboundSender._build_file_chain_result(
            text=reply.content, file_paths=file_paths, event=event,
        )
    else:
        result = event.plain_result(reply.content)
    event.set_result(result)
    event.stop_event()
    logger.info(
        "outbound: content=%.50s, files=%d, conversation=%s",
        reply.content, len(file_paths), reply.conversation_id,
    )
    @staticmethod
    async def send_files(
        file_paths: list[dict],
        event: "AstrMessageEvent",
        caption: str = "",
    ) -> None:
        try:
            result = AstrBotOutboundSender._build_file_chain_result(
                text=caption, file_paths=file_paths, event=event,
            )
            await event.send(result)
            logger.info(
                "outbound send_files: %d file(s), caption=%.50s",
                len(file_paths), caption,
            )
        except Exception as e:
            logger.warning("M13 send_files failed: %s", e)
    @staticmethod
    def _build_file_chain_result(
        text: str,
        file_paths: list[dict],
        event: "AstrMessageEvent",
    ):
        from astrbot.core.message.components import Plain, File, Image
        chain = []
        if text:
            chain.append(Plain(text))
        for fp in file_paths:
            path = fp.get("path", "")
            name = fp.get("name", "")
```

```

        file_type = fp.get("type", "file")
        if not path:
            continue
        if file_type == "image":
            chain.append(Image(file=path))
        else:
            chain.append(File(name=name or path.rsplit("/", 1)[-1], file=path))
    return event.chain_result(chain)
@staticmethod
async def send_progress(text: str, event: "AstrMessageEvent") -> None:
    try:
        msg = event.plain_result(text)
        await event.send(msg)
        logger.info(
            "outbound progress: text=%.50s",
            text,
        )
    except Exception as e:
        logger.warning("M8b progress direct send failed, falling back: %s", e)
        result = event.plain_result(text)
        event.set_result(result)

```

文件编号：213 文件路径：data/plugins/emily_agent/adapters/sse_listener.py

```

from __future__ import annotations
import asyncio
import json
import logging
from typing import TYPE_CHECKING
import aiohttp
if TYPE_CHECKING:
    from .astrbot.outbound_sender import AstrBotOutboundSender
    from .standard.reply import ReplyMessage
logger = logging.getLogger("emily.plugin.sse")
class SSEListener:
    def __init__(self, outbound: "AstrBotOutboundSender", event_registry: dict | None = None):
        self.outbound = outbound
        self._event_registry = event_registry if event_registry is not None else {}
        self._running = False
        self._handlers = {
            "reply": self._handle_reply,
            "progress": self._handle_progress,
            "file_send": self._handle_file_send,
            "session_closed": self._handle_session_closed,
        }

```

```
async def listen(self, sse_url: str, reconnect_delay: float = 3.0) -> None:
    self._running = True
    while self._running:
        try:
            await self._listen_once(sse_url)
        except Exception as e:
            logger.warning("SSE connection lost: %s — reconnecting in %.0fs", e, reconnect_delay)
            await asyncio.sleep(reconnect_delay)
    def stop(self) -> None:
        self._running = False
    async def _listen_once(self, sse_url: str) -> None:
        async with aiohttp.ClientSession() as session:
            async with session.get(sse_url) as resp:
                logger.info("SSE connected: %s (status=%d)", sse_url, resp.status)
                event_type = "message"
                data_buf: list[str] = []
                async for raw in resp.content:
                    line = raw.decode("utf-8", errors="ignore").rstrip("/r/n")
                    if line == "":
                        if data_buf:
                            await self._dispatch(event_type, "/n".join(data_buf))
                            event_type = "message"
                            data_buf = []
                            continue
                    if line.startswith(":"):
                        continue
                    if line.startswith("event:"):
                        event_type = line[len("event:"):].strip()
                    elif line.startswith("data:"):
                        data_buf.append(line[len("data:"):].strip())
    async def _dispatch(self, event_type: str, data_str: str) -> None:
        try:
            data = json.loads(data_str) if data_str else {}
        except json.JSONDecodeError:
            logger.warning("SSE bad data for %s: %s", event_type, data_str[:120])
            return
        handler = self._handlers.get(event_type)
        if handler:
            try:
                await handler(data)
            except Exception as e:
                logger.warning("SSE handler '%s' failed: %s", event_type, e)
        else:
            logger.debug("SSE unhandled event type: %s", event_type)
```

```
async def _handle_reply(self, data: dict) -> None:
    from .standard.reply import ReplyMessage
    conv_id = data.get("conversation_id", "")
    event = self._event_registry.get(conv_id)
    if event is None:
        logger.debug("SSE reply: no event for conv=%s (already replied sync?)", conv_id)
        return
    reply = ReplyMessage(
        conversation_id=conv_id,
        content=data.get("content", ""),
        reply_to_message_id=data.get("reply_to_message_id"),
    )
    await self.outbound.send(reply, event)
async def _handle_progress(self, data: dict) -> None:
    conv_id = data.get("conversation_id", "")
    event = self._event_registry.get(conv_id)
    text = data.get("content", "")
    if event is not None and text:
        await self.outbound.send_progress(text, event)
async def _handle_file_send(self, data: dict) -> None:
    conv_id = data.get("conversation_id", "")
    event = self._event_registry.get(conv_id)
    if event is None:
        return
    file_paths = data.get("file_paths", [])
    caption = data.get("caption", "")
    if file_paths:
        await self.outbound.send_files(file_paths=file_paths, event=event, caption=caption)
async def _handle_session_closed(self, data: dict) -> None:
    conv_id = data.get("conversation_id", "")
    self._event_registry.pop(conv_id, None)
    logger.debug("SSE session_closed: conv=%s", conv_id)
```

文件编号: 214 文件路径: data/plugins/emily_agent/adapters/standard/command.py

```
from dataclasses import dataclass
@dataclass
class EventCommand:
    project_id: str | None = None
    project_name: str | None = None
    title: str = ""
    event_type: str = "general"
    category: str = "待分类"
    description: str = ""
    event_date: str | None = None
```

```
creator_id: str = ""
source_message_id: str = ""
related_event_ids: List[str] | None = None
conversation_id: str = ""
```

@dataclass

class TaskCommand:

```
project_id: str | None = None
project_name: str | None = None
title: str = ""
description: str = ""
assignee_text: str = ""
due_date: str | None = None
due_text: str = ""
creator_id: str = ""
source_message_id: str = ""
```

@dataclass

class MeetingCommand:

```
project_id: str | None = None
project_name: str | None = None
title: str = ""
summary: str = ""
attendees: List[str] | None = None
creator_id: str = ""
source_message_id: str = ""
```

@dataclass

class FileCommand:

```
project_id: str | None = None
project_name: str | None = None
filename: str = ""
file_type: str = ""
file_size: int = 0
storage_path: str = ""
uploaded_by: str = ""
source_message_id: str = ""
```

@dataclass

class QueryCommand:

```
query_type: str = "event"
project_id: str | None = None
project_name: str | None = None
time_range: str = "all"
status_filter: str | None = None
assignee: str | None = None
sender_name: str | None = None
keyword: str | None = None
```



```
intent: str | None = None
file_type: str | None = None
conversation_id: str | None = None
limit: int = 50
```

文件编号：215 文件路径：data/plugins/emily_agent/adapters/standard/message.py

```
from dataclasses import dataclass, field
from datetime import datetime
@dataclass
class StandardMessage:
    message_id: str = ""
    platform: str = ""
    conversation_type: str = ""
    conversation_id: str = ""
    sender_id: str = ""
    sender_name: str = ""
    group_id: str | None = None
    group_name: str | None = None
    content: str = ""
    is_at_bot: bool = False
    mentioned_user_ids: list[str] = field(default_factory=list)
    reply_to_message_id: str | None = None
    timestamp: datetime | None = None
    raw_event: dict | None = None
    msg_type: int = 1
    attachments: list[dict] = field(default_factory=list)
```

文件编号：216 文件路径：data/plugins/emily_agent/adapters/standard/reply.py

```
from dataclasses import dataclass, field
@dataclass
class ReplyMessage:
    conversation_id: str
    content: str
    reply_to_message_id: str | None = None
    references: list[dict] = field(default_factory=list)
    metadata: dict = field(default_factory=dict)
    file_paths: list[dict] = field(default_factory=list)
```

文件编号：217 文件路径：data/plugins/emily_agent/adapters/standard/result.py

```
from dataclasses import dataclass, field
@dataclass
class RouteResult:
    intent: str
    project_id: str | None = None
```

```
project_name: str | None = None
confidence: float = 0.0
data: dict = field(default_factory=dict)
```

```
@dataclass
```

```
class HandlerResult:
    success: bool
    object_type: str | None = None
    object_id: str | None = None
    reply: str | None = None
    error_code: str | None = None
    pending_confirmation: bool = False
```

```
@dataclass
```

```
class AgentStep:
    step_index: int
    type: str
    detail: str
    timestamp: str = ""
```

```
@dataclass
```

```
class AgentResult:
    success: bool
    reply: str
    steps: list[AgentStep] = field(default_factory=list)
    error_code: str | None = None
```

文件编号：218 文件路径：data/plugins/emily_agent/adapters/standard/route_decision.py

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class RouteDecision:
    takeover: bool
    mode: str = "collaborate"
    intent: str | None = None
    confidence: float = 0.0
    handler: str | None = None
    should_reply: bool = True
    reason: str = ""
```

文件编号：219 文件路径：data/plugins/emily_agent/main.py

```
import asyncio
import hashlib
import logging
from collections import deque
from sys import maxsize
from astrbot.api import star
from astrbot.api.event import AstrMessageEvent, filter
```

```
from .adapters.astrbot.inbound_adapter import AstrBotInboundAdapter
from .adapters.astrbot.outbound_sender import AstrBotOutboundSender
from .adapters.api_client import EmilyApiClient
from .adapters.sse_listener import SSEListener
logger = logging.getLogger("emily.plugin")
DEDUP_MAX = 200

def _event_fingerprint(event: AstrMessageEvent) -> str:
    raw = f"{event.session_id}|{event.message_str}"
    return hashlib.sha256(raw.encode()).hexdigest()[:16]

class Main(star.Star):
    def __init__(self, context: star.Context, config: dict | None = None) -> None:
        super().__init__(context, config=config)
        cfg = dict(config) if config else {}
        base_url = cfg.get("emycore_url", "http://emily-core:18080")
        api_token = cfg.get("emycore_api_token", "")
        self.inbound = AstrBotInboundAdapter()
        self.outbound = AstrBotOutboundSender()
        self.api = EmilyApiClient(base_url=base_url, api_token=api_token)
        self._event_registry: dict = {}
        self.sse = SSEListener(self.outbound, event_registry=self._event_registry)
        self._sse_url = cfg.get("emycore_sse_url", "") or self.api.get_sse_url()
        self._seen: deque[str] = deque(maxlen=DEDUP_MAX)
        self._sse_task = None

    async def initialize(self) -> None:
        self._sse_task = asyncio.create_task(self.sse.listen(self._sse_url))
        health = await self.api.health_check()
        logger.info("Emily Core health: %s", health.get("status", "?"))

    async def terminate(self) -> None:
        self.sse.stop()
        if self._sse_task is not None:
            self._sse_task.cancel()
        await self.api.close()

    @filter.event_message_type(filter.EventType.ALL, priority=maxsize)
    async def on_message(self, event: AstrMessageEvent) -> None:
        event_id = _event_fingerprint(event)
        if event_id in self._seen:
            return
        self._seen.append(event_id)
        msg = self.inbound.to_standard_message(event)
        if msg.conversation_id:
            self._event_registry[msg.conversation_id] = event
        reply = await self.api.send_message(msg)
        if reply is not None:
            await self.outbound.send(reply, event)
```

文件编号：220 文件路径：data/plugins/emily_agent/metadata.yaml

name: emily_agent

desc: "Emily 薄通信层插件 —— 消息去重 + 格式转换 + HTTP 转发到 emily-core 容器 + SSE 出站推送（业务

version: 1.0.0

author: Emily